TensorFlow 2.0

(开始时间：2021/7/28)

(by 孟智超)

#

1 CATLOG

1 CATLOG
2 一、开发环境安装
2.1 CUDA_10.0
2.2 cuDNN_7.6.5（for CUDA 10.0）
2.3 环境变量
2.4 安装tensorflow 2.0 gpu版本
2.5 tensorflow
2.5.1 自动求导
2.5.2 GPU加速
3 二、线性回归
3.1 梯度下降（Gradient Descent）
3.2 梯度下降求解
3.2.1 损失函数求值
3.2.2 梯度下降更新w，b
3.2.3 设置迭代次数，计算停止迭代后的w，b
3.3 梯度下降实战
4 三、手写数字识别
5 四、TensorFlow基础
5.1 数据类型
5.1.1 数值类型(int,float,double)
5.1.2 字符串类型
5.1.3 布尔类型
5.2 数值精度
5.2.1 指定精度(int*,float*,double)
5.2.2 读取精度
5.2.3 类型转换
5.3 待优化张量
5.4 创建张量
5.4.1 从数组、列表对象创建
5.4.2 创建全0或全1张量(zeros、ones)
5.4.3 创建自定义数值张量(fill)
5.4.4 创建已知分布的张量(normal、uniform)
5.4.5 创建序列(tf.range)
5.5 张量的典型应用
5.5.1 标量（scalar）
5.5.2 向量(vector)
5.5.3 矩阵(matrix)
5.5.4 三维张量(tensor)
5.5.5 四维张量(tensor)

2 一、开发环境安装

2.1 CUDA_10.0

- **下载地址**

> https://developer.nvidia.com/cuda-toolkit-archive

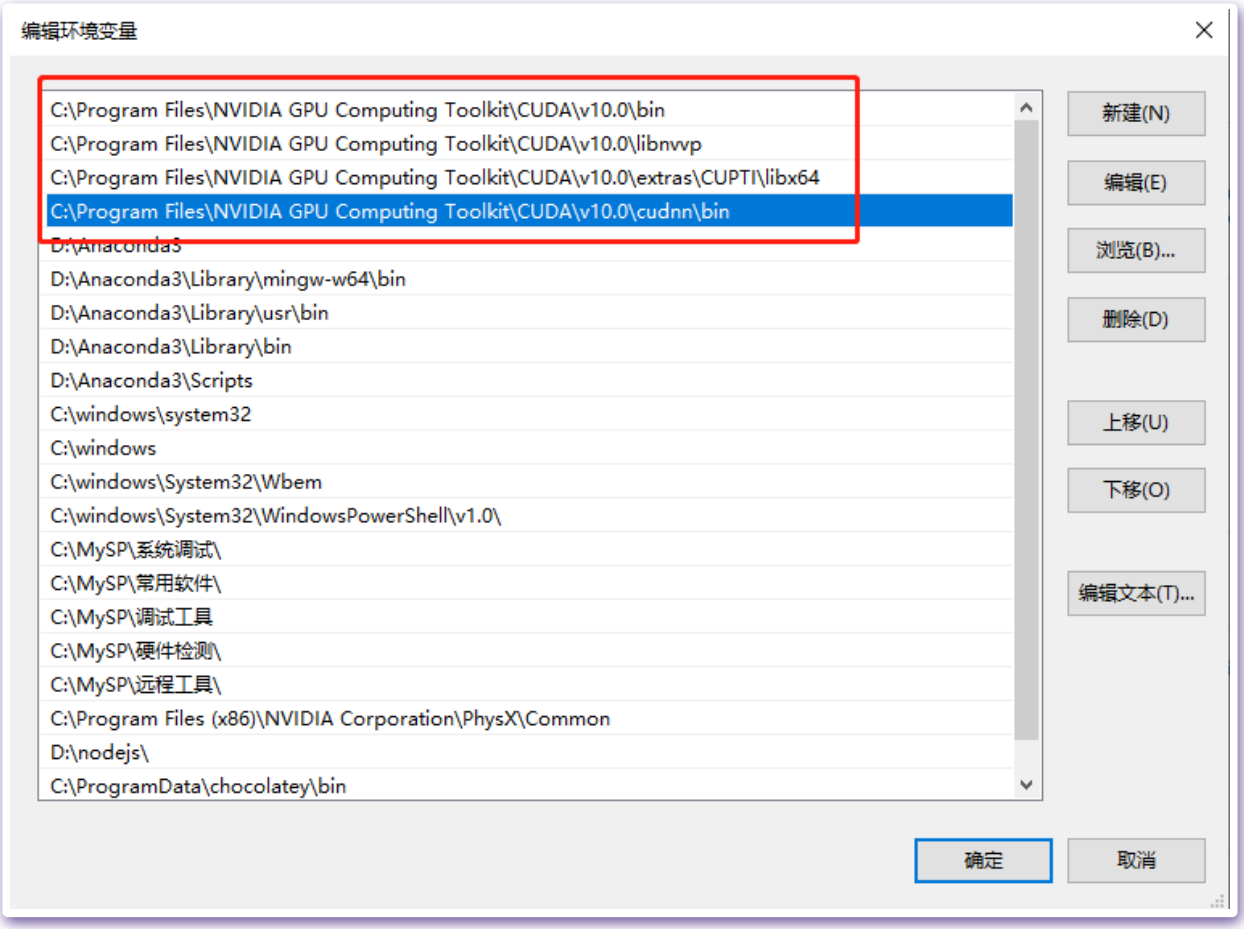
2.2 cuDNN_7.6.5（for CUDA 10.0）

- **下载地址**

> https://developer.nvidia.com/rdp/cudnn-archive

2.3 环境变量

- 四个置顶



2.4 安装tensorflow 2.0 gpu版本

- 命令行安装

```
pip install tensorflow-gpu==2.0.0
```

- `a = tf.constant(2)`
  `b = tf.constant(4)`
  `a*b` 查看是否成功

```
import tensorflow as tf
tf.test.is_gpu_available()

"""运行结果"""
True
```

```
a = tf.constant(2)
b = tf.constant(4)

a*b

"""运行结果"""
<tf.Tensor: id=2, shape=(), dtype=int32, numpy=8>

```

2.5 tensorflow

2.5.1 自动求导

- 以 $y = aw^2 + bw + c$ 为例

$$y = aw^2 + bw + c \tag{1}$$

$$\frac{\partial y}{\partial w} = 2aw + b \tag{2}$$

1. a = 1
2. b=2
3. c=3
4. w=4

$$\frac{\partial y}{\partial w} = 2aw + b = 2 * 4 + 2 = 10 \tag{3}$$

```python
import tensorflow as tf

# 创建4个张量
a = tf.constant(1.)
b = tf.constant(2.)
c = tf.constant(3.)
w = tf.constant(4.)

with tf.GradientTape() as tape:  # 构建梯度环境
    tape.watch([w])  # 将w加入梯度跟踪列表
    # 构建计算过程
    y = a * w ** 2 + b * w + c
# 求导
[dy_dw] = tape.gradient(y, [w])
print(dy_dw)

```

运行结果：

```
tf.Tensor(10.0, shape=(), dtype=float32)
```

2.5.2 GPU加速

-

3 二、线性回归

3.1 梯度下降（Gradient Descent）

- 线性回归损失函数

$$Loss = \frac{1}{n}\sum_{i=1}^{n}(w * x_i + b - y_i)^2 \tag{4}$$

- `w,b` 的t梯度
    1. `w` 的梯度

$$\frac{\partial Loss}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}(w * x_i + b - y_i) * x_i \tag{5}$$

    2. `b` 的梯度

$$\frac{\partial Loss}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(w * x_i + b - y_i) \tag{6}$$

3.2 梯度下降求解

3.2.1 损失函数求值

- 代码

```python
def compute_error_for_line_given_points(b, w, points):
    """
    calculate the loss function
    :param b: bias
    :param w: weight
    :param points:  data_point
    :return: current value of loss function
    """
    totalError = 0
    for i in range(0, len(points)):
        x = points[i, 0]
        y = points[i, 1]
        # compute mean_squared_error
        totalError += (y - (w * x + b)) ** 2
    # average loss for each point
    return totalError / float(len(points))

```

3.2.2 梯度下降更新w，b

- 代码

```
# gradient descent
def step_gradient(b_current, w_current, points, learningRate):
    """update w,b vias gradient descent"""
    b_gradient = 0
    w_gradient = 0
    N = float(len(points))
    for i in range(0, len(points)):
        x = points[i, 0]
        y = points[i, 1]
        # grad_b = 2(wx+b-y)
        b_gradient += (2 / N) * ((w_current * x + b_current) - y)
        # grad_w = 2(wx+b-y)x
        w_gradient += (2 / N) * ((w_current * x + b_current) - y) * x
    # update w'    b'
    new_b = b_current - (learningRate * b_gradient)
    new_w = w_current - (learningRate * w_gradient)

    return [new_b, new_w]
```

3.2.3 设置迭代次数，计算停止迭代后的w，b

- 代码

```
def gradient_descent_runner(points, starting_b, starting_w, learning_rate, num_iterations):
    """
    iterat gradient descent
    :param points: data_points
    :return: updated b,w
    """
    b = starting_b
    w = starting_w
    # update for several times
    for i in range(num_iterations):
        b, w = step_gradient(b, w, np.array(points), learning_rate)
    return [b, w]
```

3.3 梯度下降实战

- 全部代码

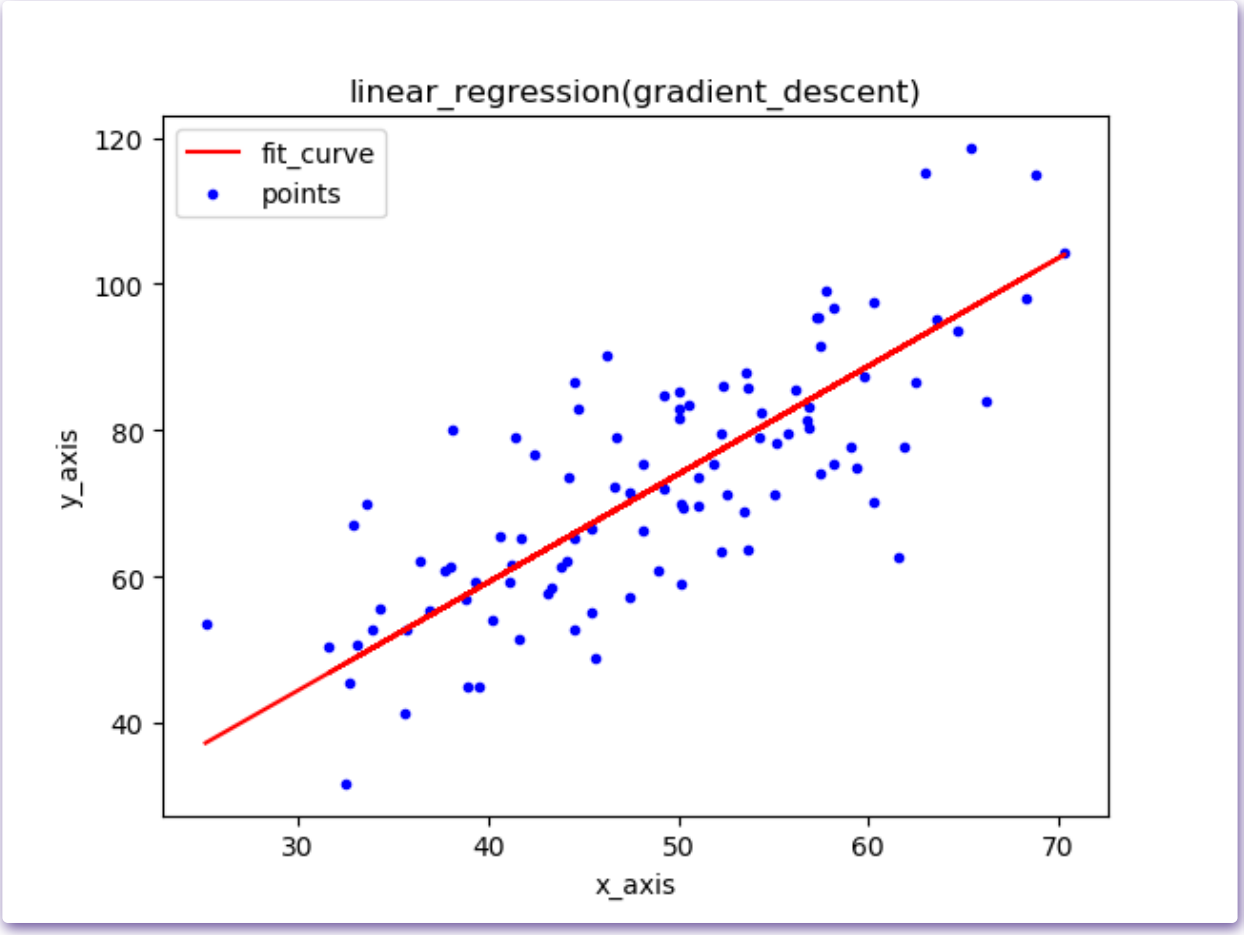
```
import numpy as np
from matplotlib import pyplot as plt


# y = wx + b
def compute_error_for_line_given_points(b, w, points):
```

```python
    """
    calculate the loss function
    :param b: bias
    :param w: weight
    :param points:  data_point
    :return: current value of loss function
    """
    totalError = 0
    for i in range(0, len(points)):
        x = points[i, 0]
        y = points[i, 1]
        # compute mean_squared_error
        totalError += (y - (w * x + b)) ** 2
    # average loss for each point
    return totalError / float(len(points))


# gradient descent
def step_gradient(b_current, w_current, points, learningRate):
    """update w,b vias gradient descent"""
    b_gradient = 0
    w_gradient = 0
    N = float(len(points))
    for i in range(0, len(points)):
        x = points[i, 0]
        y = points[i, 1]
        # grad_b = 2(wx+b-y)
        b_gradient += (2 / N) * ((w_current * x + b_current) - y)
        # grad_w = 2(wx+b-y)x
        w_gradient += (2 / N) * ((w_current * x + b_current) - y) * x
    # update w'    b'
    new_b = b_current - (learningRate * b_gradient)
    new_w = w_current - (learningRate * w_gradient)

    return [new_b, new_w]


def gradient_descent_runner(points, starting_b, starting_w, learning_rate, num_iterations):
    """
    iterat gradient descent
    :param points: data_points
    :return: updated b,w
    """
    b = starting_b
    w = starting_w
    # update for several times
    for i in range(num_iterations):
        b, w = step_gradient(b, w, np.array(points), learning_rate)
    return [b, w]

```

```python

def run():
    points = np.genfromtxt("./need/data.csv", delimiter=",")
    learning_rate = 0.0001
    initial_b = 0  # initial y-intercept guess
    initial_w = 0  # initial slope guess
    num_iterations = 1000
    print("Starting gradient descent at\n b = {0}, w = {1}, error = {2}"
          .format(initial_b, initial_w,
                  compute_error_for_line_given_points(initial_b, initial_w, points))
          )
    print("Running...")
    [b, w] = gradient_descent_runner(points, initial_b, initial_w, learning_rate, num_iterations)
    print("After {0} iterations\n b = {1}, w = {2}, error = {3}".
          format(num_iterations, b, w,
                 compute_error_for_line_given_points(b, w, points))
          )

    plt.scatter(points[:, 0], points[:, 1], c='b', marker='.')
    plt.plot(points[:, 0], points[:, 0] * w + b, c='r')
    plt.title('linear_regression(gradient_descent)')
    plt.xlabel('x_axis')
    plt.ylabel('y_axis')
    plt.legend(['fit_curve', 'points'])
    plt.show()


if __name__ == '__main__':
    run()

```

运行结果：

```
Starting gradient descent at
 b = 0, w = 0, error = 5565.107834483211
Running...
After 1000 iterations
 b = 0.08893651993741344, w = 1.4777440851894448, error = 112.61481011613473
```


linear_regression(gradient_descent)
fit_curve
points
120
100
80
60
40
y_axis
30
40
50
60
70
x_axis

4 三、手写数字识别

5 四、TensorFlow基础

```
import tensorflow as tf
import tensorflow.keras as keras
import tensorflow.keras.layers as layers

physical_devices = tf.config.experimental.list_physical_devices('GPU')
assert len(physical_devices) > 0, "Not enough GPU hardware devices available"
tf.config.experimental.set_memory_growth(physical_devices[0], True)
```

5.1 数据类型

5.1.1 数值类型(int,float,double)

- 标量在 TensorFlow 是如何创建的

```
# python 语言方式创建标量
a = 1.2
# TF 方式创建标量
aa = tf.constant(1.2)

type(a), type(aa), tf.is_tensor(aa)
```

输出如下：

```
(float, tensorflow.python.framework.ops.EagerTensor, True)
```

- 如果要使用 TensorFlow 提供的功能函数， 须通过 TensorFlow 规定的方式去创建张量，而不能使用 Python 语言的标准变量创建方式。

```
x = tf.constant([1,2.,3.3])
# 打印 TF 张量的相关信息
x
```

输出如下：

```
<tf.Tensor: id=1, shape=(3,), dtype=float32, numpy=array([1. , 2. , 3.3], dtype=float32)>
```

- 将 TF 张量的数据导出为 numpy 数组格式

```
# 将 TF 张量的数据导出为 numpy 数组格式
x.numpy()
```

输出如下：

```
array([1. , 2. , 3.3], dtype=float32)
```

- 与标量不同，向量的定义须通过 List 容器传给 tf.constant()函数。

  创建一个元素的向量：

```
# 创建一个元素的向量
a = tf.constant([1.2])
a, a.shape
```

输出如下：

```
(<tf.Tensor: id=2, shape=(1,), dtype=float32, numpy=array([1.2],
dtype=float32)>,
 TensorShape([1]))
```

- 创建 3 个元素的向量：

```
 # 创建 3 个元素的向量
a = tf.constant([1,2, 3.])
a, a.shape
```

输出如下：

```
(<tf.Tensor: id=3, shape=(3,), dtype=float32, numpy=array([1., 2.,
3.], dtype=float32)>,
 TensorShape([3]))
```

- 定义矩阵

```
# 创建 2 行 2 列的矩阵
a = tf.constant([[1,2],[3,4]])
a, a.shape
```

输出如下：

```
(<tf.Tensor: id=4, shape=(2, 2), dtype=int32, numpy=
 array([[1, 2],
        [3, 4]])>, TensorShape([2, 2]))
```

- 三维张量可以定义为：

```
# 创建 3 维张量
tf.constant([[[1,2],[3,4]],[[5,6],[7,8]]])
```

输出如下：

```
<tf.Tensor: id=5, shape=(2, 2, 2), dtype=int32, numpy=
array([[[1, 2],
        [3, 4]],

       [[5, 6],
        [7, 8]]])>
```

- 通过传入字符串对象即可创建字符串类型的张量

```
# 创建字符串
a = tf.constant('Hello, Deep Learning.')
a
```

输出如下：

```
<tf.Tensor: id=6, shape=(), dtype=string, numpy=b'Hello, Deep Learning.'>
```

5.1.2 字符串类型

- 通过传入字符串对象即可创建字符串类型的张量

```
# 创建字符串
a = tf.constant('Hello, Deep Learning.')
a
```

输出如下：

```
<tf.Tensor: id=7, shape=(), dtype=string, numpy=b'Hello, Deep Learning.'>
```

- 在 tf.strings 模块中，提供了常见的字符串类型的工具函数，如小写化 lower()、 拼接 join()、 长度 length()、 切分 split()等。

```
# 小写化字符串
tf.strings.lower(a)
```

输出如下：

```
<tf.Tensor: id=8, shape=(), dtype=string, numpy=b'hello, deep learning.'>
```

5.1.3 布尔类型

- 布尔类型的张量只需要传入 Python 语言的布尔类型数据，转换成 TensorFlow 内部布尔型即可。

```
# 创建布尔类型标量
tf.constant(True)
```

输出如下：

```
<tf.Tensor: id=9, shape=(), dtype=bool, numpy=True>
```

- 创建布尔类型的向量

```
 # 创建布尔类型向量
tf.constant([True, False])
```

输出如下：

```
<tf.Tensor: id=10, shape=(2,), dtype=bool, numpy=array([ True,
False])>
```

- 需要注意的是， TensorFlow 的布尔类型和 Python 语言的布尔类型并不等价，不能通用

```
# 创建 TF 布尔张量
a = tf.constant(True)
# TF 布尔类型张量与 python 布尔类型比较
print(a is True)
# 仅数值比较
print(a == True)
```

输出如下：

```
False

tf.Tensor(True, shape=(), dtype=bool)
```

5.2 数值精度

5.2.1 指定精度(int*,float*,double)

- 在创建张量时，可以指定张量的保存精度

  $2^{16} = 65536$

  如用 `int16` 保存123456789会有溢出，因为最大保存为$2^{16} - 1$

```
# 创建指定精度的张量
a=tf.constant(123456789, dtype=tf.int16)
b=tf.constant(123456789, dtype=tf.int32)
print(a)
print(b)
```

输出如下：

```
<tf.Tensor: id=12, shape=(), dtype=int16, numpy=-13035>
<tf.Tensor: id=15, shape=(), dtype=int32, numpy=123456789>
```

- 对于浮点数， 高精度的张量可以表示更精准的数据，例如采用 tf.float32 精度保存π时，实际保存的数据为 3.1415927

```
import numpy as np
# 从 numpy 中导入 pi 常量
print(np.pi)
# 32 位
tf.constant(np.pi, dtype=tf.float32)
```

输出如下：

```
3.141592653589793
<tf.Tensor: id=16, shape=(), dtype=float32, numpy=3.1415927>
```

如果采用 `tf.float64` (即 `tf.double`) 精度保存π，则能获得更高的精度

```
tf.constant(np.pi, dtype=tf.float64) # 64 位
```

输出如下：

```
<tf.Tensor: id=17, shape=(), dtype=float64, numpy=3.141592653589793>
```

5.2.2 读取精度

- 通过访问张量的 dtype 成员属性可以判断张量的保存精度

```
a = tf.constant(np.pi, dtype=tf.float16)

# 读取原有张量的数值精度
print('before:',a.dtype)
# 如果精度不符合要求，则进行转换
if a.dtype != tf.float32:
    # tf.cast 函数可以完成精度转换
    a = tf.cast(a,tf.float32)
# 打印转换后的精度
print('after :',a.dtype)
```

输出如下：

```
before: <dtype: 'float16'>
after : <dtype: 'float32'>
```

5.2.3 类型转换

- 系统的每个模块使用的数据类型、 数值精度可能各不相同， 对于不符合要求的张量的类型及精度， 需要通过 `tf.cast` 函数进行转换

```
# 创建 tf.float16 低精度张量
a = tf.constant(np.pi, dtype=tf.float16)
# 转换为高精度张量
tf.cast(a, tf.double)
```

输出如下：

```
<tf.Tensor: id=21, shape=(), dtype=float64, numpy=3.140625>
```

- 进行类型转换时，需要保证转换操作的合法性， 例如将高精度的张量转换为低精度的张量时，可能发生数据**溢出**隐患 ：

```
a = tf.constant(123456789, dtype=tf.int32)
# 转换为低精度整型
tf.cast(a, tf.int16)
```

输出如下：

```
<tf.Tensor: id=23, shape=(), dtype=int16, numpy=-13035>
```

- 布尔类型与整型之间相互转换也是合法的， 是比较常见的操作

```
a = tf.constant([True, False])
# 布尔类型转整型
tf.cast(a, tf.int32)
```

输出如下：

```
<tf.Tensor: id=25, shape=(2,), dtype=int32, numpy=array([1, 0])>
```

一般默认 0 表示 False， 1 表示 True，在 TensorFlow 中，将非 0 数字都视为 True，

```
a = tf.constant([-1, 0, 1, 2])
# 整型转布尔类型
tf.cast(a, tf.bool)
```

输出如下：

```
<tf.Tensor: id=27, shape=(4,), dtype=bool, numpy=array([ True,
False,  True,  True])>
```

5.3 待优化张量

- `TensorFlow` 增加了一种专门的数据类型来支持梯度信息的记录： `tf.Variable` 。tf.Variable 类型在普通的张量类型基础上添加了 `name` ， `trainable` 等属性来支持计算图的构建。

```
# 创建 TF 张量
a = tf.constant([-1, 0, 1, 2])
# 转换为 Variable 类型
aa = tf.Variable(a)
# Variable 类型张量的属性
aa.name, aa.trainable
```

输出如下：

```
('Variable:0', True)
```

  1. `name` 属性用于命名计算图中的变量，这套命名体系是 `TensorFlow` 内部维护的， 一般不需要用户关注 `name` 属性；
  2. `trainable` 属性表征当前张量是否需要被优化，创建 `Variable` 对象时是默认启用优化标志，可以设置 `trainable=False` 来设置张量不需要优化。

- 直接创建张量

```
# 直接创建 Variable 张量
tf.Variable([[1,2],[3,4]])
```

输出如下：

```
<tf.Variable 'Variable:0' shape=(2, 2) dtype=int32, numpy=
array([[1, 2],
       [3, 4]])>
```

5.4 创建张量

5.4.1 从数组、列表对象创建

- 通过 `tf.convert_to_tensor` 函数可以创建新 `Tensor` ，并将保存在 `Python List` 对象或者 `Numpy Array` 对象中的数据导入到新 `Tensor` 中。

```
# 从列表创建张量
tf.convert_to_tensor([1,2.])
```

输出如下：

```
<tf.Tensor: id=44, shape=(2,), dtype=float32, numpy=array([1., 2.],
dtype=float32)>
```

```
# 从数组中创建张量
tf.convert_to_tensor(np.array([[1,2.],[3,4]]))
```

输出如下：

```
<tf.Tensor: id=45, shape=(2, 2), dtype=float64, numpy=
array([[1., 2.],
       [3., 4.]])>
```

5.4.2 创建全0或全1张量(zeros、ones)

5.4.2.1 Scalar

- 创建全 0，全 1 的标量

```
# 创建全 0，全 1 的标量
tf.zeros([]),tf.ones([])
```

输出如下：

```
(<tf.Tensor: id=46, shape=(), dtype=float32, numpy=0.0>,
 <tf.Tensor: id=47, shape=(), dtype=float32, numpy=1.0>)
```

5.4.2.2 Vector

- 创建全 0，全 1 的向量

```
# 创建全 0，全 1 的向量
tf.zeros([1]),tf.ones([1])
```

输出如下：

```
(<tf.Tensor: id=50, shape=(1,), dtype=float32, numpy=array([0.],
dtype=float32)>,
 <tf.Tensor: id=53, shape=(1,), dtype=float32, numpy=array([1.],
dtype=float32)>)
```

5.4.2.3 Matrix

- 创建全 0 的矩阵

```
# 创建全 0 矩阵，指定 shape 为 2 行 2 列
tf.zeros([2,2])
```

输出如下：

```
<tf.Tensor: id=56, shape=(2, 2), dtype=float32, numpy=
array([[0., 0.],
       [0., 0.]], dtype=float32)>
```

- 创建全 1 的矩阵

```
# 创建全 1 矩阵，指定 shape 为 3 行 2 列
tf.ones([3,2])
```

输出如下：

```
<tf.Tensor: id=59, shape=(3, 2), dtype=float32, numpy=
array([[1., 1.],
       [1., 1.],
       [1., 1.]], dtype=float32)>
```

5.4.2.4 zeors_like、ones_like

- 通过 `tf.zeros_like, tf.ones_like` 可以方便地新建与某个张量 `shape` 一致， 且内容为全 0 或全 1 的张量。

```
# 创建一个矩阵
a = tf.ones([2,3])
# 创建一个与 a 形状相同，但是全 0 的新矩阵
tf.zeros_like(a)
```

输出如下：

```
<tf.Tensor: id=63, shape=(2, 3), dtype=float32, numpy=
array([[0., 0., 0.],
       [0., 0., 0.]], dtype=float32)>
```

创建与张量A形状一样的全 1 张量

```
# 创建一个矩阵
a = tf.zeros([3,2])
# 创建一个与 a 形状相同，但是全 1 的新矩阵
tf.ones_like(a)
```

输出如下：

```
<tf.Tensor: id=69, shape=(3, 2), dtype=float32, numpy=
array([[1., 1.],
       [1., 1.],
       [1., 1.]], dtype=float32)>
```

5.4.3 创建自定义数值张量(fill)

- 通过 `tf.fill(shape, value)` 可以创建全为自定义数值 `value` 的张量，形状由 `shape` 参数指定。

- 创建-1 的标量

```
# 创建-1 的标量
tf.fill([], -1)
```

输出如下：

```
<tf.Tensor: id=72, shape=(), dtype=int32, numpy=-1>
```

- 创建-1 的向量

```
# 创建-1 的向量
tf.fill([1], -1)
```

输出如下：

```
<tf.Tensor: id=75, shape=(1,), dtype=int32, numpy=array([-1])>
```

- 创建 2 行 2 列，元素全为 99 的矩阵

```
# 创建 2 行 2 列，元素全为 99 的矩阵
tf.fill([2,2], 99)
```

输出如下：

```
<tf.Tensor: id=78, shape=(2, 2), dtype=int32, numpy=
array([[99, 99],
       [99, 99]])>
```

5.4.4 创建已知分布的张量(normal、uniform)

5.4.4.1 正态分布

- 通过 `tf.random.normal(shape, mean=0.0, stddev=1.0)` 可以创建形状为 `shape`，均值为 `mean`，标准差为 `stddev` 的 正态分布 $\mathcal{N}(mean, stddev^2)$。

- 创建标准正态分布的张量

```
# 创建标准正态分布的张量
tf.random.normal([2,2])
```

输出如下：

```
<tf.Tensor: id=84, shape=(2, 2), dtype=float32, numpy=
array([[0.4479265 , 1.2336508 ],
       [0.96864706, 2.076528  ]], dtype=float32)>
```

- 创建均值为 1，标准差为 2 的正态分布的张量

```
# 创建均值为 1，标准差为 2 的正态分布的张量
tf.random.normal([2,2], mean=1,stddev=2)
```

输出如下：

```
<tf.Tensor: id=90, shape=(2, 2), dtype=float32, numpy=
array([[ 0.06462109,  1.8899262 ],
       [-1.3979323 ,  0.07801807]], dtype=float32)>
```

5.4.4.2 均匀分布

- 通过 `tf.random.uniform(shape, minval=0, maxval=None, dtype=tf.float32)` 可以创建采样自 `[minval, maxval)` 区间的均匀分布的张量

- 创建采样自[0,1)均匀分布的矩阵

```
# 创建采样自[0,1)均匀分布的矩阵
tf.random.uniform([3,2])
```

输出如下：

```
<tf.Tensor: id=97, shape=(3, 2), dtype=float32, numpy=
array([[0.5197921 , 0.80691314],
       [0.38051474, 0.05250001],
       [0.18329549, 0.8741617 ]], dtype=float32)>
```

- 创建采样自[0,10)均匀分布的矩阵

```
# 创建采样自[0,10)均匀分布的矩阵
tf.random.uniform([2,2],maxval=10)
```

输出如下：

```
<tf.Tensor: id=104, shape=(2, 2), dtype=float32, numpy=
array([[0.4374528 , 7.3671246 ],
       [2.1262336 , 0.12584329]], dtype=float32)>
```

- 如果需要均匀采样整形类型的数据，必须指定采样区间的最大值 maxval 参数，同时指定数据类型为 `tf.int*` 型

```
# 创建采样自[0,100)均匀分布的整型矩阵
tf.random.uniform([2,2],maxval=100,dtype=tf.int32)
```

输出如下：

```
<tf.Tensor: id=108, shape=(2, 2), dtype=int32, numpy=
array([[15, 50],
       [38, 84]])>
```

5.4.5 创建序列(tf.range)

- `tf.range(limit, delta=1)` 可以创建 `[0, limit)` 之间，步长为 `delta` 的整型序列，不包含 `limit` 本身。

- 创建0~10，不包含 10，默认步长为1的序列

```
# 0~10，不包含 10
tf.range(10)
```

输出如下：

```
<tf.Tensor: id=112, shape=(10,), dtype=int32, numpy=array([0, 1, 2,
3, 4, 5, 6, 7, 8, 9])>
```

- 创建 0~10，步长为 2 的整形序列

```
# 创建 0~10，步长为 2 的整形序列
tf.range(10,delta=2)
```

输出如下：

```
<tf.Tensor: id=116, shape=(5,), dtype=int32, numpy=array([0, 2, 4,
6, 8])>
```

- 创建 1~10，步长为 2 的整形序列

```
tf.range(1,10,delta=2) # 1~10
```

```
<tf.Tensor: id=120, shape=(5,), dtype=int32, numpy=array([1, 3, 5, 7, 9])>
```

5.5 张量的典型应用

5.5.1 标量（scalar）

- 标量，如模型的损失函数值

```
# 随机模拟网络输出
out = tf.random.uniform([4,10])
# 随机构造样本真实标签
y = tf.constant([2,3,2,0])
# one-hot 编码
y = tf.one_hot(y, depth=10)
# 计算每个样本的 MSE
loss = tf.keras.losses.mse(y, out)
# 平均 MSE,loss 应是标量
loss = tf.reduce_mean(loss)
print(loss)
```

输出如下：

```
tf.Tensor(0.28147948, shape=(), dtype=float32)
```

5.5.2 向量(vector)

- 如$WX+b$中的b

- 考虑 2 个输出节点的网络层， 我们创建长度为 2 的偏置向量b，并累加在每个输出节点上：

```
# z=wx,模拟获得激活函数的输入 z
z = tf.random.normal([4,2])
# 创建偏置向量
b = tf.zeros([2])
# 累加上偏置向量
z = z + b
z
```

输出如下：

```
<tf.Tensor: id=147, shape=(4, 2), dtype=float32, numpy=
array([[-2.004388  , -0.30366704],
       [ 0.5251593 ,  0.61763936],
       [ 1.1223725 , -2.277926  ],
       [-0.20396625, -0.31192765]], dtype=float32)>
```

- 创建输入节点数为 4，输出节点数为 3 的线性层网络，那么它的偏置向量 b 的长度应为 3

```
# 创建一层 Wx+b，输出节点为 3
fc = tf.keras.layers.Dense(3)
# 通过 build 函数创建 W,b 张量，输入节点为 4
fc.build(input_shape=(2,4))
# 查看偏置向量
fc.bias
```

输出如下：

```
<tf.Variable 'bias:0' shape=(3,) dtype=float32, numpy=array([0., 0., 0.], dtype=float32)>
```

5.5.3 矩阵(matrix)

- 矩阵，如$WX+b$中的W

```
# 2 个样本，特征长度为 4 的张量
x = tf.random.normal([2,4])
# 定义 W 张量
w = tf.ones([4,3])
# 定义 b 张量
b = tf.zeros([3])
# X@W+b 运算
o = x@w+b
o
```

输出如下：

```
<tf.Tensor: id=185, shape=(2, 3), dtype=float32, numpy=
array([[ 3.4116697,  3.4116697,  3.4116697],
       [-1.0482166, -1.0482166, -1.0482166]], dtype=float32)>
```

```
# 定义全连接层的输出节点为 3
fc = tf.keras.layers.Dense(3)
# 定义全连接层的输入节点为 4
fc.build(input_shape=(2,4))
# 查看权值矩阵 W
fc.kernel
```

输出如下：

```
<tf.Variable 'kernel:0' shape=(4, 3) dtype=float32, numpy=
array([[-0.00832933,  0.3129425 ,  0.05450684],
       [ 0.4455037 ,  0.8942138 ,  0.19051397],
       [ 0.18969214, -0.43504652, -0.8300707 ],
       [ 0.6065289 ,  0.33857238, -0.15131074]], dtype=float32)>
```

5.5.4 三维张量(tensor)

- 三维张量，如电影评分

```
# 自动加载 IMDB 电影评价数据集
(x_train,y_train),(x_test,y_test)=keras.datasets.imdb.load_data(num_words=10000)
# 将句子填充、截断为等长 80 个单词的句子
x_train = keras.preprocessing.sequence.pad_sequences(x_train,maxlen=80)
x_train.shape
```

输出如下：

```
(25000,80)
```

可以看到 `x_train` 张量的 `shape` 为 `[25000,80]`，其中 `25000` 表示句子个数， `80` 表示每个句子共 80 个单词，每个单词使用数字编码方式表示。

我们通过 layers.Embedding 层将数字编码的单词转换为长度为 100 个词向量：

```
# 创建词向量 Embedding 层类
embedding = tf.keras.layers.Embedding(10000, 100)
# 将数字编码的单词转换为词向量
out = embedding(x_train)
out.shape
```

输出如下：

```
TensorShape([25000, 80, 100])
```

可以看到，经过 Embedding 层编码后，句子张量的 shape 变为[25000,80,100]，其中 100 表示每个单词编码为长度是 100 的向量。

5.5.5 四维张量(tensor)

- 如图片识别

```
# 创建 32x32 的彩色图片输入，个数为 4
x = tf.random.normal([4,32,32,3])
# 创建卷积神经网络
layer = layers.Conv2D(16, kernel_size=3)
# 前向计算
out = layer(x)
# 输出大小
out.shape
```

输出如下：

```
TensorShape([4, 30, 30, 16])
```

```
# 访问卷积核张量
layer.kernel.shape
```

输出如下：

```
TensorShape([3, 3, 3, 16])
```